SYSTEM ARCHITECTURE & UML DESIGN SPECIFICATION

HELPBOT: TECHNICAL MODELING PACKAGE

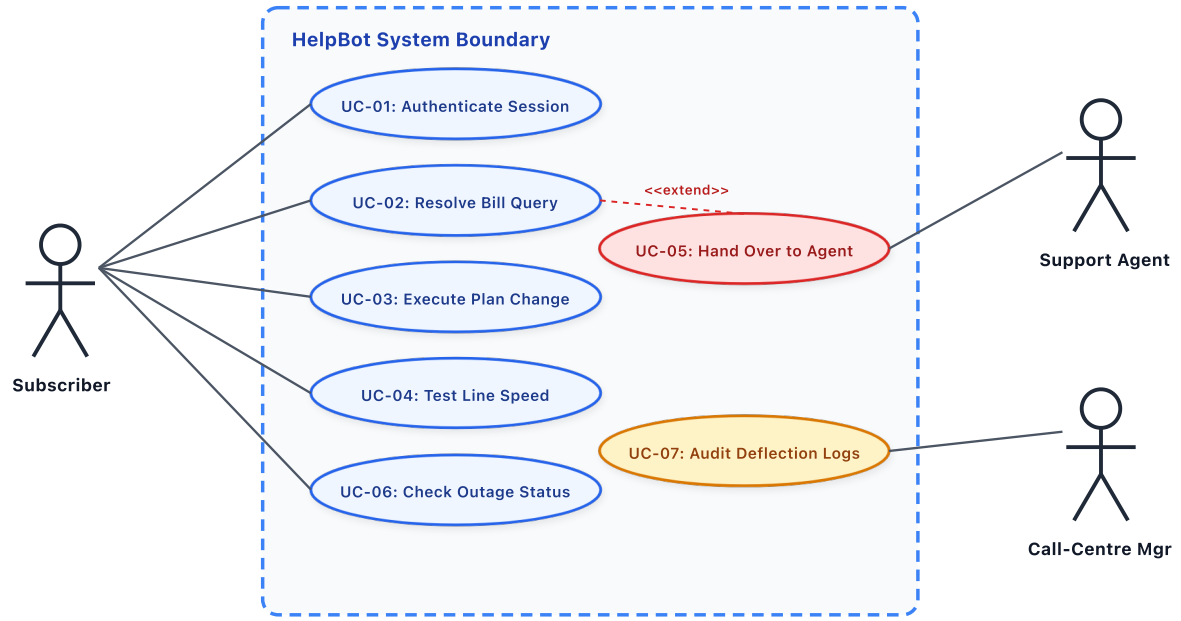
Loose Coupling Architecture, Cohesive Modular Structure & Behavioral Interaction Flows

1. Document Control & Architectural Invariants

Project Reference	Case Study No. 97 — Regional Broadband Service Provider	**UML Standard**	OMG UML 2.5 Specification
Architecture Pattern	Microservices Gateway with Out-of-Process Adapters	**Target Ingress**	WhatsApp Business API & Web Self-Care Portal

2. Actor Hierarchy & Use Case Model

The Use Case diagram encapsulates the operational boundaries of HelpBot across subscribers, support agents, call-centre managers, and external enterprise backends.

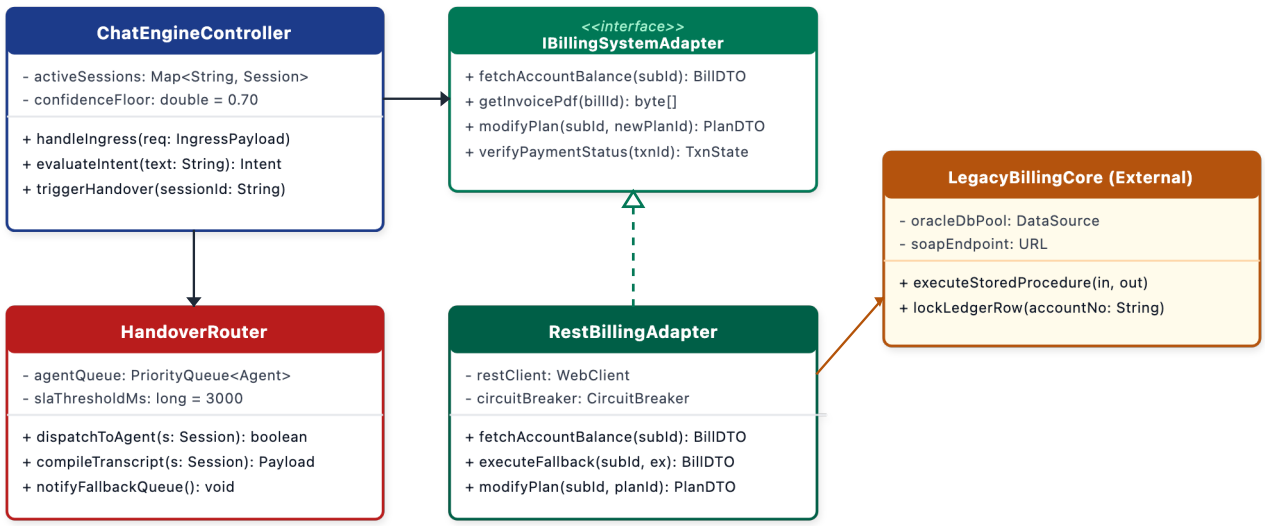


3. Traceability of Use Cases to Functional Scope

Use Case ID	Use Case Title	Primary Actor	System Boundary Outcome
UC-01	Authenticate Session	Subscriber	Validates subscriber via RMN & OTP; initializes session context.
UC-02	Resolve Bill Query	Subscriber	Fetches outstanding ledger balance, dueDate, and invoice PDF.
UC-03	Execute Plan Change	Subscriber	Presents catalogue; provisions upgrade with dual confirmation.
UC-04	Test Line Speed	Subscriber	Probes OLT line power, executes sync check, advises router reboot.
UC-05	Hand Over to Agent	Support Agent	Transfers session context and full transcript in ≤ 3.0s.
UC-06	Check Outage Status	Subscriber	Matches physical node ID to active NOC incidents.
UC-07	Audit Deflection Logs	Call-Centre Manager	Monitors resolution rate, intent accuracy, and agent deflection.

4. Structural Class Hierarchy (Loosely Coupled Architecture)

The structural model isolates the core conversational engine from legacy provider billing and network systems using the **Adapter Design Pattern** and **Resilience4j Circuit Breakers**.



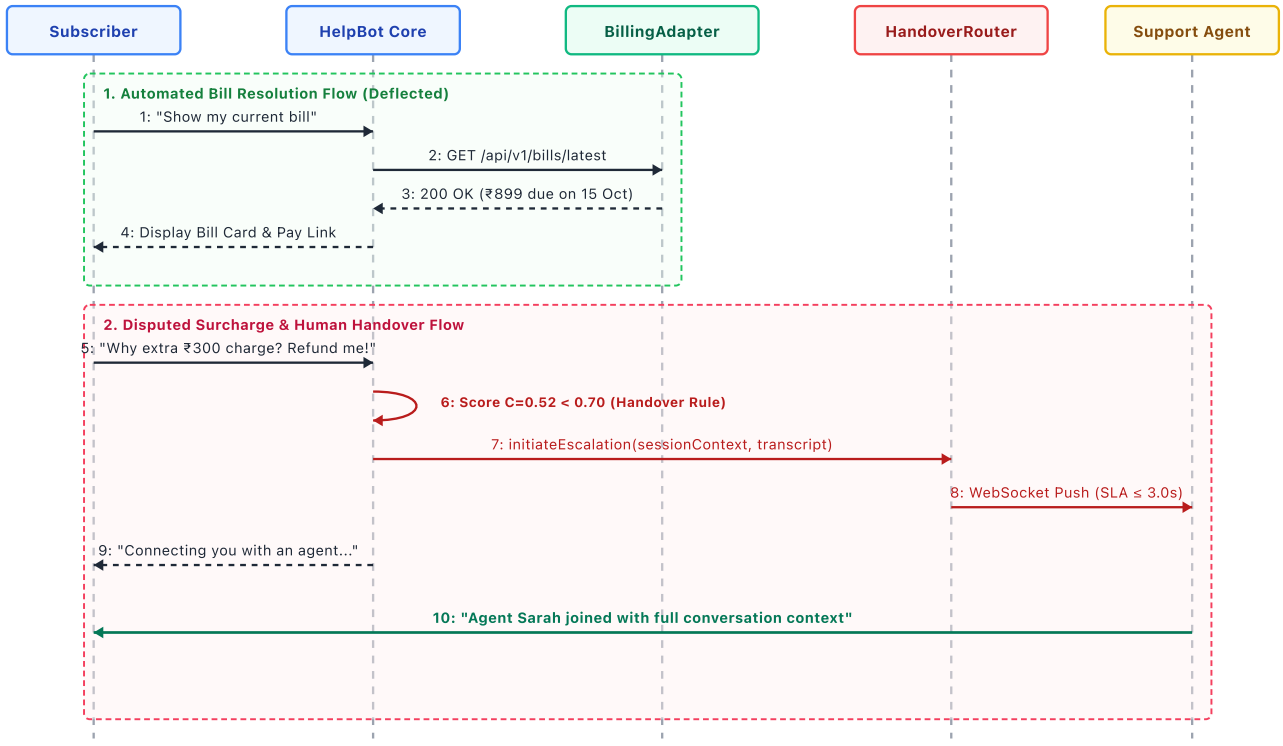
5. Architectural Cohesion & Loose Coupling Defense

> **Separation of Concerns:** As dictated by the project objectives, the chatbot conversational runtime and the provider's billing core must remain strictly loosely coupled with high module cohesion.

Design Principle	Implementation Mechanism	SEPM Quality Assurance Benefit
High Functional Cohesion	Single Responsibility Principle (SRP): `ChatEngineController` handles dialogue states only; `HandoverRouter` strictly coordinates agent queues; `RestBillingAdapter` translates protocols.	Eliminates ripple effects. Altering billing response schemas requires zero changes in conversational dialogue state algorithms.
Loose Coupling via Adapters	The chatbot engine communicates exclusively via the abstract interface `IBillingSystemAdapter`. It possesses zero direct JDBC or Oracle database connections into the billing system.	If the legacy billing system experiences a database deadlock or maintenance outage, the chatbot does not crash; it fails over gracefully.
Fault Isolation (Resilience4j)	All external REST integrations utilize a circuit breaker configured with a 3.0-second timeout threshold and automated half-open retry states.	Satisfies non-functional requirement NFR-06, ensuring external billing latency never cascades into conversational dropouts.

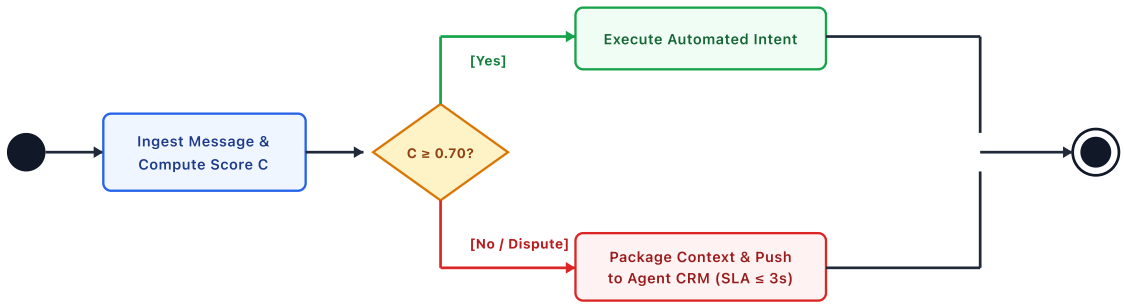
6. Sequence Diagram: Resolve Bill Query & Hand Over to Human Agent

The sequence model illustrates end-to-end messaging for two critical operational flows: successful retrieval of billing data, followed by an escalated transaction requiring human handover due to low confidence (C < 0.70).



7. Activity Diagram: Conversation Control & Handover Decision Logic

Models deterministic routing across automated diagnostic branches, threshold checks, and fallback mechanisms.



8. State Machine Diagram: Chat Conversation & Session Lifecycle

Specifies explicit, deterministic transitions governing active customer support sessions.

